THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 14

Unity Catalog Fundamentals — Databricks' Governance Backbone

SECTION 01

The Problem Unity Catalog Solves

Before Unity Catalog, each Databricks workspace maintained its own separate Hive metastore. This meant data governance was fragmented: permissions, lineage, and access control had to be managed independently per workspace, with no unified view across an organization.

- Data silos across workspaces: each workspace's Hive metastore meant data wasn't accessible across different workspaces without extra setup.
- Fragmented governance: teams often stitched together third-party tools like Apache Ranger to fill security gaps that weren't natively addressed.
- Limited cross-cloud portability: governance models didn't extend cleanly across regions or cloud providers.

CORE CONCEPT

Unity Catalog centralizes governance for structured and unstructured data, Delta Lake and Apache Iceberg tables, machine learning models, notebooks, dashboards, and files, replacing the patchwork of workspace-level metastores and third-party governance tools with a single, purpose-built layer.

SECTION 02

The Object Hierarchy

Unity Catalog's structure is hierarchical, and this hierarchy is the foundation for how both organization and access control work.

Level	What It Is
Metastore	The top-level container for data in Unity Catalog. One metastore can be created per region and attached to any number of workspaces in that region.
Catalog	The primary unit of data isolation. Catalogs typically map to organizational units, data domains, or SDLC scopes (for example, production vs development).
Schema	Exists within a catalog; often referred to as a database. A catalog can contain multiple schemas.
Table / View / Volume / Function / Model	The actual securable objects within a schema; the third level of the namespace.

SECTION 03

The Three-Level Namespace

CORE CONCEPT

Unity Catalog supports a three-level namespace, catalog.schema.table, across workspaces and clouds,

with SQL-based access control that any data practitioner familiar with ANSI SQL can pick up quickly.

```
-- Fully qualified three-level reference
SELECT * FROM production.sales.transactions;

--          ^catalog  ^schema  ^table

-- Setting context so you don't repeat the full path
USE CATALOG production;
USE SCHEMA sales;
SELECT * FROM transactions;
```

SECTION 04

What Happens to the Old Hive Metastore

Unity Catalog doesn't force teams to migrate everything overnight. The legacy per-workspace Hive metastore continues to be accessible.

GOOD TO KNOW

The Hive metastore appears as a top-level catalog called hive_metastore within Unity Catalog's three-level namespace. This means old and new governance models can coexist, and by using three-level namespace notation, you can even join data in a Unity Catalog metastore with data in the legacy Hive metastore.

SECTION 05

How Access Control Follows the Hierarchy

Because Unity Catalog is hierarchical, permissions cascade through it. Having access to a table alone is not enough.

IMPORTANT NUANCE

USE CATALOG and USE SCHEMA permissions are required on the catalog and schema for all operations on objects inside them. Regardless of a principal's privileges on a specific table, they must also have USE CATALOG on its parent catalog and USE SCHEMA on its parent schema just to reach that table. This trips up a lot of people troubleshooting 'permission denied' errors for the first time.

SECTION 06

Managed Tables vs External Tables

Table Type	Storage Location	Who Manages Lifecycle
Managed Table	Determined by Unity Catalog, though the	Unity Catalog handles both

	data still lives in your cloud account	governance and the underlying file storage lifecycle
External Table	Specified explicitly by you	Unity Catalog handles governance only; you manage the storage lifecycle

Databricks recommends using managed tables where possible, in order to take advantage of the latest table features and simplified lifecycle management.

SECTION 07

The Unity Catalog Object Model

```
METASTORE (one per region, shared across workspaces)
     |
     +-- CATALOG: production
     |      +-- SCHEMA: sales
     |      |      +-- TABLE: transactions
     |      |      +-- VIEW: monthly_summary
     |      +-- SCHEMA: marketing
     |             +-- TABLE: campaigns
     |
     +-- CATALOG: development
     |      +-- SCHEMA: sandbox
     |
     +-- CATALOG: hive_metastore  (legacy, auto-surfaced)
```

REMEMBER THIS FOR INTERVIEWS

Unity Catalog's structure is metastore, then catalog, then schema, then the actual object. Permissions cascade down this same hierarchy, which is why access issues often trace back to a missing USE CATALOG or USE SCHEMA grant, not the table-level permission itself.

SECTION 08

Glossary — Day 14 Terms

Term	Definition
Metastore	The top-level container for data in Unity Catalog, shareable across multiple workspaces in a region.
Catalog	The primary unit of data isolation, typically mapped to an organizational unit or SDLC scope.
Schema	A container within a catalog, often referred to as a database, holding tables,

	views, and other objects.
Securable Object	Any object in Unity Catalog on which permissions can be granted, such as a catalog, schema, table, or model.
Three-Level Namespace	The catalog.schema.table addressing scheme used to reference any data asset in Unity Catalog.
hive_metastore	The special catalog name under which the legacy, workspace-level Hive metastore is surfaced within Unity Catalog.
Managed Table	A table whose storage location and lifecycle are handled by Unity Catalog, though data still resides in your cloud account.

SECTION 09

Interview Questions & Answers

Q1. What problem was Unity Catalog designed to solve?

A. Before Unity Catalog, each Databricks workspace had its own separate Hive metastore, causing data silos and fragmented governance across workspaces. Unity Catalog centralizes governance for data, AI assets, and files into a single layer shareable across workspaces and clouds.

Q2. What is the three-level namespace in Unity Catalog?

A. catalog.schema.table (or more broadly catalog.schema.object). Catalog is the first layer, schema is the second, and the actual securable object like a table, view, or model is the third.

Q3. What happens to the legacy Hive metastore after adopting Unity Catalog?

A. It doesn't disappear. It appears as a special top-level catalog called hive_metastore within Unity Catalog's own three-level namespace, allowing old and new data to coexist and even be joined together.

Q4. If a user has SELECT on a table, is that enough to query it?

A. Not necessarily. The user must also have USE CATALOG on the table's parent catalog and USE SCHEMA on its parent schema, since these permissions are required for all operations on objects inside them, regardless of table-level privileges.

Q5. What is the difference between a managed table and an external table in Unity Catalog?

A. For a managed table, Unity Catalog determines the storage location and handles both governance and the storage lifecycle. For an external table, you specify the storage location yourself, and Unity Catalog only handles governance, not the storage lifecycle.

Q6. Can a single Unity Catalog metastore be shared across multiple workspaces?

A. Yes. One metastore can be created per region and attached to any number of workspaces in that region, giving every linked workspace the same view of the governed data.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab